

TaybGo Driver

Backend Requirements for Accept / Reject Action Buttons from Push Notifications

Prepared for backend and mobile implementation planning ? Last updated: April 25, 2026

This document defines what the backend must provide so a driver can accept or reject an order from a notification while the app is closed, backgrounded, or locked. The mobile app owns the native notification UI and action handlers; the backend owns the offer lifecycle, validation, assignment, race-safety, and source-of-truth order state.

Important principle: a notification action should never be treated as a simple update to an order id. It should answer a short-lived, driver-specific order offer that the backend can validate atomically.

1. Scope

The feature allows drivers to respond to an incoming order offer directly from the operating system notification surface. The expected actions are Accept and Reject. The action may be triggered without opening the full Flutter UI.

- ? Backend sends actionable order-offer push data to one or more eligible drivers.
- ? Mobile renders an OS notification with Accept and Reject actions using the received payload.
- ? When an action is tapped, mobile calls an authenticated backend decision endpoint.
- ? Backend validates the offer, applies the decision, updates order assignment, and emits follow-up events.
- ? Backend responses must be deterministic for expired, cancelled, already accepted, or repeated requests.

2. Required Backend Capabilities

Capability	Backend requirement
Order offer record	Create a unique offer/assignment candidate for each driver notification. Do not rely on order_id alone.
Push payload	Send offer_id, order_id, expiry, service type, display summary, and platform-specific notification metadata.
Decision endpoint	Expose one endpoint for Accept/Reject decisions from notification handlers and in-app screens.
Atomic assignment	Accept must be processed inside a transaction or equivalent lock so only one driver can win.
Idempotency	Repeated taps, retries, or network replays must return a stable current result.
Expiry	Offer validity and push TTL must match. Old notifications must not allow stale acceptance.
Token registry	Store FCM/APNs-capable device tokens per driver and platform; remove invalid tokens.
Audit trail	Record who decided, when, from which device, and whether the source was notification or in-app.

3. Recommended End-to-End Flow

Step	Actor	Action
1	Backend	Find eligible driver(s) for a new order.
2	Backend	Create an order_offer row for each driver with status=pending and expires_at.
3	Backend	Send FCM/APNs push containing offer_id, order_id, expiry, category/action metadata, and display data.
4	Mobile	Displays notification with Accept and Reject buttons registered locally/native-side.
5	Driver	Taps Accept or Reject outside the app.
6	Mobile	Background action handler calls the decision endpoint with driver auth and offer_id.
7	Backend	Validates auth, offer ownership, expiry, order availability, and current offer state.
8	Backend	Accept: atomically assigns order to driver. Reject: marks offer rejected and continues dispatch.
9	Backend	Returns canonical status and sends follow-up events/pushes to affected clients.

4. Data Model Requirements

4.1 order_offers / assignment_candidates

Backend should maintain a driver-specific offer record. Suggested fields:

Field	Purpose
id / offer_id	Unique id used by notification payload and decision endpoint.
order_id	Order being offered.
driver_id	Only this driver may act on the offer.
status	pending, accepted, rejected, expired, superseded, cancelled.
expires_at	Server-side deadline for accepting or rejecting the offer.
created_at / responded_at	Lifecycle timestamps.
notification_message_id	Optional FCM/APNs message id for delivery tracing.
decision_source	notification_action, app_screen, backend_timeout, dispatcher, etc.
reject_reason	Optional normalized reason such as driver_rejected, timeout, unavailable.
version / lock token	Optional optimistic concurrency support.

4.2 driver_device_tokens

Backend must be able to target the correct devices and keep push delivery healthy.

- ? Store token, driver_id, platform, device_id, app_version, locale, last_seen_at, revoked_at.
- ? Support multiple active devices per driver unless product rules forbid it.
- ? Remove or disable tokens that FCM/APNs reports as invalid or unregistered.
- ? Clear or revoke tokens on logout when the app tells backend that the device is no longer authorized.

5. Push Notification Contract

Use a data payload as the reliable source for the mobile handler. The notification title/body can be included for display, but the app must receive enough structured data to call the decision endpoint without fetching first.

```
{
  "type": "order_offer",
  "offer_id": "off_7Tq91",
  "order_id": "ord_10542",
  "service_type": "food_delivery",
  "expires_at": "2026-04-25T12:30:00Z",
  "pickup_name": "TaybGo Restaurant",
  "pickup_area": "Al-Mazzeh",
  "dropoff_area": "Abu Rummaneh",
  "estimated_earning": "8.50",
  "currency": "USD",
  "distance_km": "4.2",
  "decision_token": "optional-short-lived-signed-token"
}
```

5.1 Platform metadata

? iOS: APNs payload should include a category matching the app registration, for example ORDER_OFFER. The app registers Accept/Reject actions for that category.

? Android: if using Flutter local notifications, backend sends FCM data and mobile creates the actionable local notification. If using native remote notifications, include the agreed channel/category metadata.

? Set push TTL to the same or shorter duration than expires_at, usually 30-60 seconds for dispatch offers.

? Use high priority for active dispatch offers, while still respecting platform delivery limits.

? Use collapse keys carefully. Do not collapse two different active offers into one ambiguous notification.

Do not include sensitive customer details in the notification payload unless the product explicitly allows that information on a locked screen. Prefer area-level summaries until the order is accepted.

6. Decision API Contract

The mobile app should call the same backend endpoint for notification actions and in-app buttons. This keeps validation and business rules centralized.

```
POST /api/driver/order-offers/{offerId}/decision
Authorization: Bearer <driver_jwt>
Idempotency-Key: <uuid generated by device>           // recommended
X-Device-Id: <stable device id>                       // recommended
Content-Type: application/json

{
  "decision": "accept",                                // accept | reject
  "source": "notification_action",                     // notification_action | app
  "client_action_at": "2026-04-25T12:29:42Z",
  "reject_reason": null
}
```

6.1 Success response examples

```
// Accepted
{
  "status": "accepted",
  "order_id": "ord_10542",
  "offer_id": "off_7Tq91",
  "assignment_id": "asg_5531",
  "next_action": "open_order_details"
}

// Rejected
{
  "status": "rejected",
  "order_id": "ord_10542",
  "offer_id": "off_7Tq91",
  "next_action": "dismiss_notification"
}
```

6.2 Conflict/current-state response examples

```

{
  "status": "already_taken",
  "order_id": "ord_10542",
  "message": "This order was accepted by another driver."
}

{
  "status": "expired",
  "order_id": "ord_10542",
  "message": "This offer has expired."
}

```

HTTP	Recommended meaning
200	Decision accepted, or idempotent repeat of the same already-applied decision.
401	Driver token missing/expired and no valid decision_token fallback.
403	Offer does not belong to this driver or driver is not eligible.
404	Offer id does not exist or is no longer visible to this driver.
409	Order already taken, offer expired, order cancelled, or incompatible previous decision.
422	Malformed decision body or unsupported decision value.

7. Backend Decision Rules

7.1 Accept

- ? Authenticate the driver and verify the offer belongs to that driver.
- ? Reject if the offer is not pending, has expired, or the order is cancelled.
- ? Reject if the order is already assigned to another driver.
- ? Apply the assignment in a database transaction or equivalent distributed lock.
- ? Mark the accepted offer as accepted and mark competing pending offers as superseded or expired.
- ? Create or update the driver trip/order assignment record.
- ? Return enough information for mobile to open or fetch the order details screen.

7.2 Reject

- ? Authenticate the driver and verify offer ownership.
- ? If the offer is still pending, mark it rejected and record responded_at, source, and reason.
- ? Rejecting one offer should not cancel the customer order.
- ? Continue dispatch to another driver, unless business rules say the order should wait or escalate.
- ? Avoid immediately re-offering the same order to the same driver unless a cooldown/product rule allows it.

7.3 Idempotency and retries

Notification action handlers may run more than once because of double taps, network retries, process restarts, or OS delivery behavior. Backend must support safe repeated requests.

- ? Same driver + same offer + same decision should return the existing final state.
- ? Same offer with a different later decision should return a conflict once a final decision exists.
- ? Idempotency-Key is recommended, but offer state must still be the source of truth.

8. Security Requirements

- ? Never trust order_id alone. Validate offer_id, driver_id, order state, expiry, and authorization.

- ? Notification payloads can be seen on locked screens and may be logged by devices. Keep PII minimal.
- ? Use HTTPS only. Treat notification decisions as privileged driver actions.
- ? If normal JWT refresh is unreliable from background notification handlers, provide a short-lived decision_token bound to offer_id + driver_id + expiry. It must not grant broader API access.
- ? Record audit logs for accepted/rejected decisions, including source, device id, app version if available, and server received time.
- ? Rate-limit invalid decision attempts to reduce abuse and noisy retries.

9. Follow-up Events and Client Sync

After the backend applies a decision, it should synchronize all affected clients so stale notifications and screens do not linger.

- ? Accepted: notify customer/store/dispatcher systems that a driver was assigned.
- ? Accepted: tell other offered drivers that the order is no longer available, preferably with a silent update or state event.
- ? Rejected: continue dispatch and optionally send an acknowledgement to the rejecting driver.
- ? Expired/cancelled: send or expose state so mobile can dismiss/update the notification.
- ? Return canonical order state or provide a follow-up GET endpoint that mobile can call immediately after acceptance.

10. Testing and Acceptance Criteria

Area	Acceptance criteria
Normal accept	Driver taps Accept from notification; backend assigns the order and mobile can open the active order.
Normal reject	Driver taps Reject; offer is marked rejected and order dispatch continues.
Race condition	Two drivers accept at nearly the same time; exactly one assignment succeeds.
Expired offer	Accept after expires_at returns expired and does not assign.
Already taken	Late accept after another driver wins returns already_taken/current state.
Duplicate taps	Repeated same action returns stable result without duplicate assignments or errors.
Auth	A driver cannot act on another driver's offer.
Token cleanup	Invalid push tokens are removed/disabled after provider feedback.
Offline/retry	If the device retries after network failure, backend idempotency prevents duplicate effects.
Privacy	Notification payload does not expose full customer address or phone before acceptance unless approved.

11. Open Product / Backend Decisions

- ? What is the offer expiry duration per service type: food, shipping, taxi?
- ? Can one driver receive multiple simultaneous offers, or should active offers be mutually exclusive?
- ? Should rejection require a reason, or is a simple reject enough?
- ? Should the backend allow notification accept with a short-lived decision token if the driver's JWT is expired?
- ? What exact customer/order details may appear on a locked-screen notification?
- ? Should backend send one offer at a time, a driver batch, or a rolling dispatch queue?

12. Backend Implementation Checklist

- ? Create order offer/assignment candidate model with server-side expiry.
- ? Add or confirm driver device token registration APIs.
- ? Build FCM/APNs sender for order_offer data payloads with TTL and platform metadata.
- ? Create POST /api/driver/order-offers/{offerId}/decision.
- ? Process accept/reject inside transactional, race-safe business logic.
- ? Add idempotency handling for duplicate action taps and retries.
- ? Return canonical statuses: accepted, rejected, expired, already_taken, cancelled, unauthorized.
- ? Emit follow-up events/pushes to customers, stores, other drivers, and driver app state channels.
- ? Add audit logging and metrics for delivery, tap, decision, conflict, expiry, and assignment latency.
- ? Add integration tests for normal flow, duplicate taps, expiry, unauthorized offer, and concurrent accepts.

Recommended backend summary: create a short-lived offer per driver, send that offer through push, accept decisions only through a validated offer decision endpoint, and make order assignment atomic and idempotent.